TERRAFORM - 06

Terraform Practical Documentation

Task 1: Execute the scripts shown in the video

Terraform Modules:
==========
Modules is a collection of configuration files in a directory.

For example if we want to create two ec2 for different environemnts
then i can create a directory for aws-instance configuration.
Example:
=======
main.tf
resource "aws+instance" "webserver"
ami = var.ami
instance_type = "t2.micro"
}

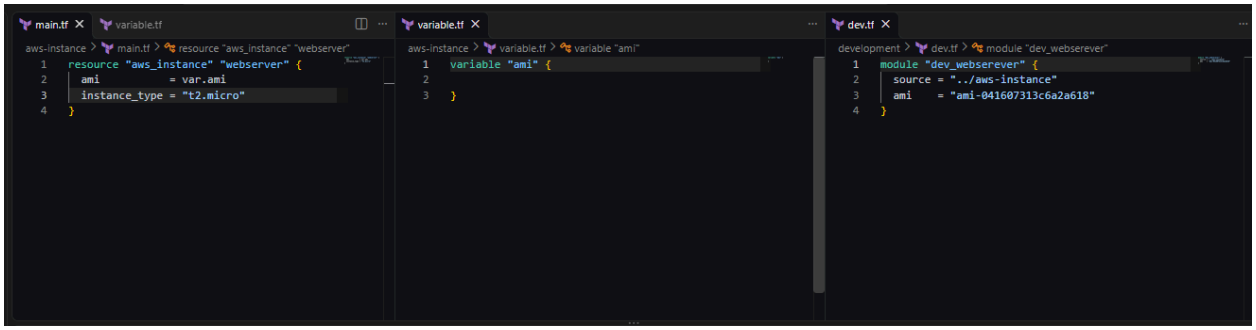
variable.tf
Variable “ami”{

}

now let me create a directory for dev under root directory.

Example:
dev.tf

```
module "dev-webserver" {
   source = "../aws_instance"
   ami = "ami-02f3f602d23f1659d"
}
```



After that go to the development directory using
Cd ./development/

```
PS C:\terraform-practice> cd .\development\
```

After do these commands
terraform init

```
PS C:\terraform-practice\development> terraform init
Initializing the backend...

Initializing modules...
- dev_webserever in ..\aws-instance

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.71.0...
- Installed hashicorp/aws v6.71.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

terraform plan

```
PS C:\terraform-practice\development> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # module.dev_webserever.aws_instance.webserver will be created
  + resource "aws_instance" "webserver" {
      + ami                                  = "ami-041607313c6a2a618"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
```

terraform apply

```
      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

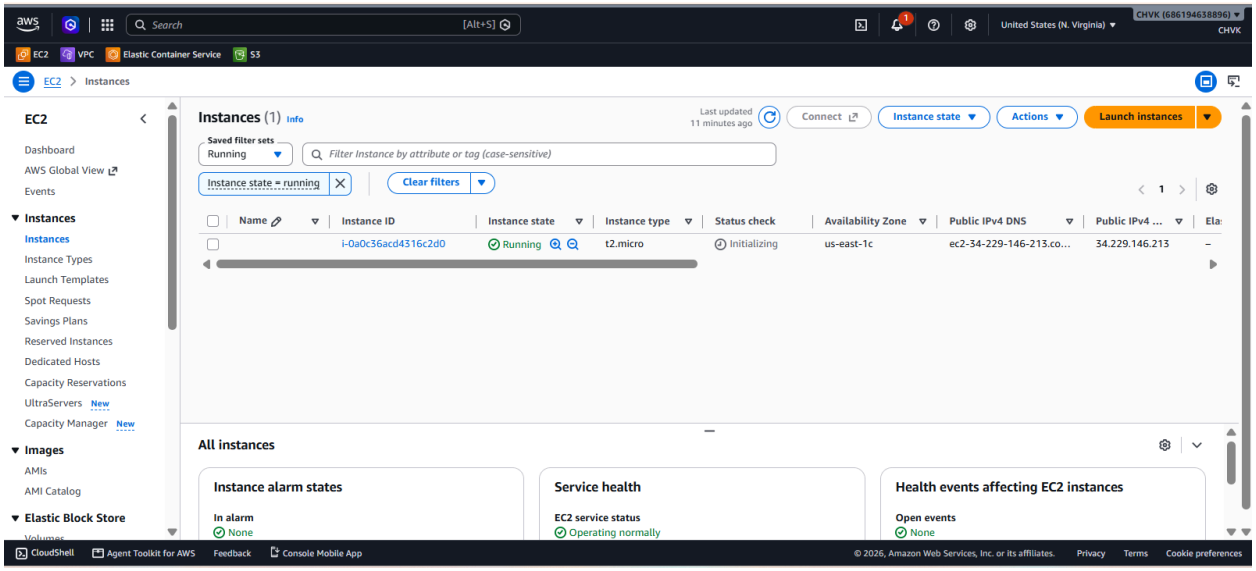
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

module.dev_webserever.aws_instance.webserver: Creating...
module.dev_webserever.aws_instance.webserver: Still creating... [00m10s elapsed]
module.dev_webserever.aws_instance.webserver: Creation complete after 18s [id=i-0a0c36acd4316c2d0]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-practice\development>
```

It will create all the statefiles and all files in the development folder
After it will create the instance
To see that instance login to your aws console and open ec2
You can see running instance



Terraform workspaces:

==========

Workspace can be used if we have multiple project/environments using the same configuration.

Workspace commands:

==========

terraform workspace new projectA --> to create a workspace names projectA
terraform workspace list --> to list the workspaces available
terraform workspace selecr projectA --> to swtich to specific workspace

```
PS C:\terraform-practice\aws-instance> terraform workspace new projectA
Created and switched to workspace "projectA"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\terraform-practice\aws-instance> terraform workspace list
  default
* projectA

PS C:\terraform-practice\aws-instance> terraform workspace select default
Switched to workspace "default".
```

Example:
main.tf:

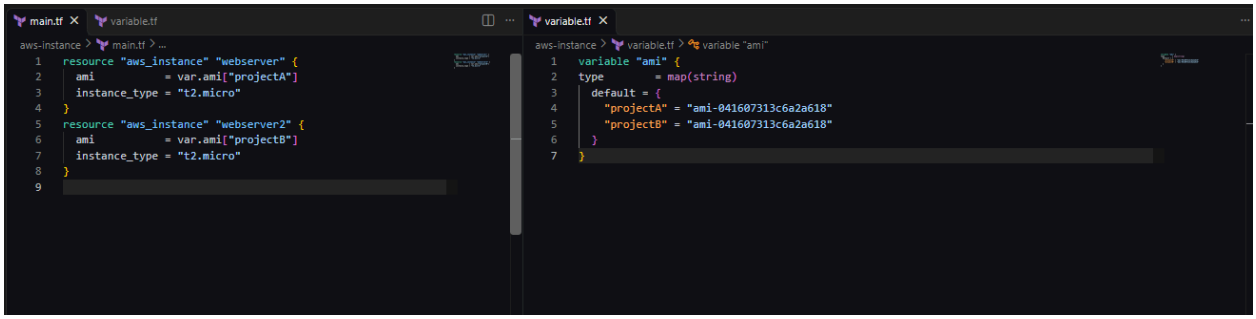
=======

```
resource "aws_instance" "webserver" {
  ami           = var.ami["projectA"]
  instance_type = "t2.micro"
}
resource "aws_instance" "webserver2" {
  ami           = var.ami["projectB"]
```

```
  instance_type = "t2.micro"
}
```

variable.tf:
==========

```
variable "ami" {
type          = map(string)
  default = {
    "projectA" = "ami-041607313c6a2a618"
    "projectB" = "ami-041607313c6a2a618"
  }
}
```



After that do these coomads
terraform init
terraform apply

```
      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

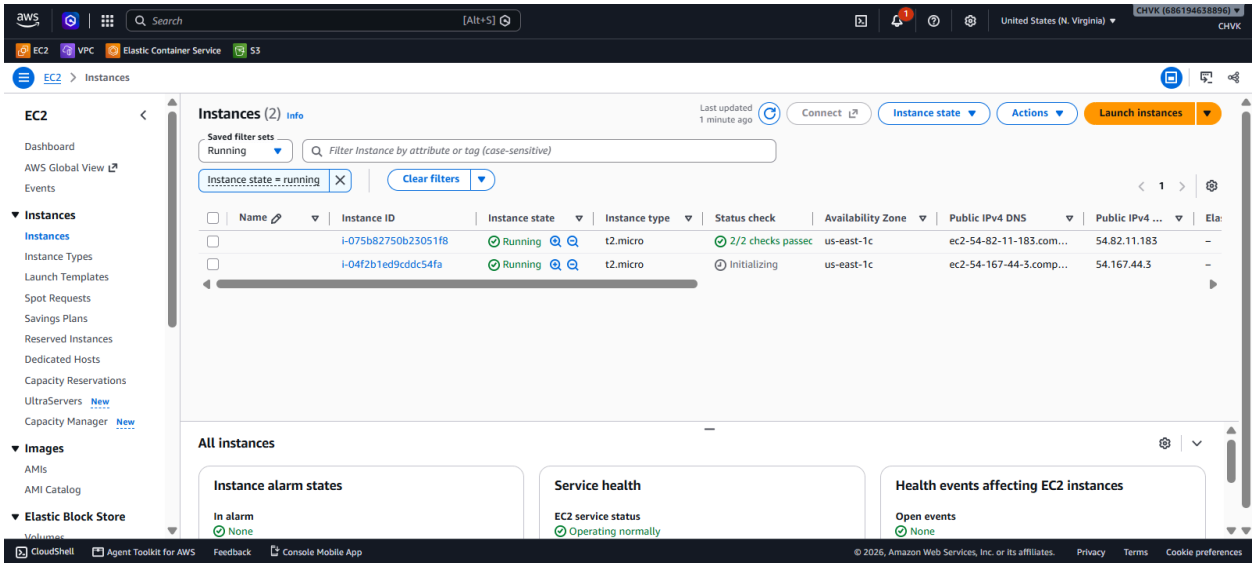
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.webserver2: Creating...
aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [00m10s elapsed]
aws_instance.webserver2: Still creating... [00m10s elapsed]
aws_instance.webserver: Creation complete after 18s [id=i-04f2b1ed9cddc54fa]
aws_instance.webserver2: Creation complete after 18s [id=i-075b82750b23051f8]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\terraform-practice\aws-instance>
```

It will creates two instances to see that go to aws console and open ec2 you can see two instances running



when using workspaces state files will be created in the main directory
under a forlder "terraform.tfstate.d"

Task 2: Provision EC2, S3, and VPC Using Terraform Modules

1. Objective

To create AWS EC2, S3, and VPC resources using Terraform modules by creating the required folders and files step by step.

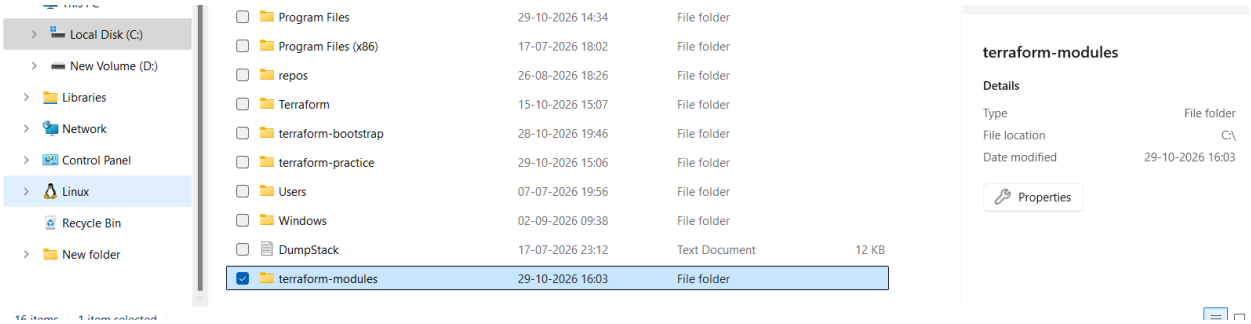
2. Prerequisites

- Windows laptop
- Visual Studio Code installed
- Terraform installed
- AWS CLI installed and configured
- AWS account with permissions to create VPC, EC2, and S3 resources

3. Step 1: Create the Main Project Folder

We will create the project in Windows.

1. Open File Explorer.
2. Open the `C:` drive.
3. Create a new folder named `terraform-modules`.
4. Open Visual Studio Code.
5. Click File → Open Folder.
6. Select `C:\terraform-modules`.
7. Click Select Folder.



Your project location is now:

`C:\terraform-modules`

4. Step 2: Create the Required Files and Directories

In VS Code, look at the Explorer panel on the left.

Revolutionise Your Coding Experience with Visual Studio Code's Game-Changing File and Folder Creation Technique | by Harsh Sharma | Medium

A. Create the `modules` folder

1. Right-click the `terraform-modules` folder.
2. Select New Folder.
3. Enter `modules` and press Enter.

Flask Tutorial in Visual Studio Code

B. Create four root files

Right-click `terraform-modules`, select New File, and create these files one by one:

- `providers.tf`
- `main.tf`

- `variables.tf`
- `outputs.tf`

C. Create three module folders

Right-click `modules`, select New Folder, and create:

- `vpc`
- `ec2`
- `s3`

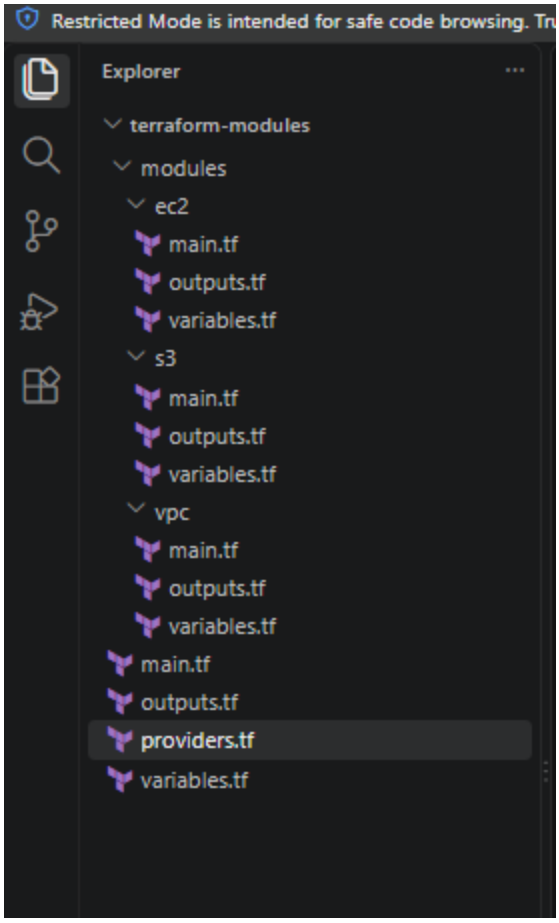
D. Create files inside each module folder

Inside each of `vpc`, `ec2`, and `s3`, create three files:

- `main.tf`
- `variables.tf`
- `outputs.tf`

Your final folder structure should look exactly like this:

```
C:\terraform-modules
│
├── providers.tf
├── main.tf
├── variables.tf
├── outputs.tf
│
└── modules
    ├── vpc
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    │
    ├── ec2
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    │
    └── s3
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```



Important: The folder names and file names must match this structure because Terraform uses the module paths in the root configuration.

5. Step 3: Add Code to `providers.tf`

Click `providers.tf` in the Explorer panel.

Copy and paste this code:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
```

```
  region = "us-east-1"
}
```

Press Ctrl + S to save the file.

Purpose: Configures Terraform to use the AWS provider in the `us-east-1` region.

6. Step 4: Add Code to the VPC Module

First, open `modules → vpc → main.tf`.

Paste this code:

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.subnet_cidr
  map_public_ip_on_launch = true

  tags = {
    Name = "terraform-public-subnet"
  }
}
```

Save the file using Ctrl + S.

Create `modules/vpc/variables.tf`

Click `modules → vpc → variables.tf` and paste:

```
variable "vpc_cidr" {
  description = "CIDR range for the VPC"
  type        = string
}
```

```
variable "subnet_cidr" {
  description = "CIDR range for the subnet"
  type        = string
}
```

Save the file.

Create `modules/vpc/outputs.tf`

Click `modules → vpc → outputs.tf` and paste:

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "subnet_id" {
  value = aws_subnet.public.id
}
```

Save the file.

Purpose: The VPC module creates a VPC and subnet. Its output file makes their IDs available to the root configuration and other modules.

7. Step 5: Add Code to the EC2 Module

Open `modules → ec2 → main.tf`.

Paste:

```
resource "aws_instance" "server" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  subnet_id              = var.subnet_id
  associate_public_ip_address = true

  tags = {
    Name = var.instance_name
  }
}
```

Save the file.

Create `modules/ec2/variables.tf`

Paste the following code:

```
variable "ami_id" {
  description = "Amazon Machine Image ID"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "subnet_id" {
  description = "Subnet ID for EC2"
  type        = string
}

variable "instance_name" {
  description = "Name of the EC2 instance"
  type        = string
}
```

Save the file.

Create `modules/ec2/outputs.tf`

Paste:

```
output "instance_id" {
  value = aws_instance.server.id
}

output "public_ip" {
  value = aws_instance.server.public_ip
}
```

Save the file.

Purpose: The EC2 module creates an instance in the subnet supplied by the VPC module. The outputs display the instance ID and assigned public IP address.

8. Step 6: Add Code to the S3 Module

Open `modules → s3 → main.tf`.

Paste:

```
resource "aws_s3_bucket" "storage" {
  bucket = var.bucket_name

  tags = {
    Name = "terraform-storage"
  }
}
```

Save the file.

Create `modules/s3/variables.tf`

Paste:

```
variable "bucket_name" {
  description = "Globally unique S3 bucket name"
  type        = string
}
```

Save the file.

Create `modules/s3/outputs.tf`

Paste:

```
output "bucket_name" {
  value = aws_s3_bucket.storage.bucket
}
```

Save the file.

Purpose: The S3 module creates a bucket using the name supplied by the root configuration. The bucket name must be globally unique across AWS accounts.

9. Step 7: Add Code to the Root `main.tf`

Now click the root `main.tf` directly under `terraform-modules`, not the one inside a module folder.

Paste the following complete code:

```
# Find the latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Create the VPC module
module "vpc" {
  source = "./modules/vpc"

  vpc_cidr    = "10.0.0.0/16"
  subnet_cidr = "10.0.1.0/24"
}

# Create the EC2 module
module "ec2" {
  source = "./modules/ec2"

  ami_id        = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
  subnet_id     = module.vpc.subnet_id
  instance_name = "Terraform-EC2"
}

# Create the S3 module
module "s3" {
  source = "./modules/s3"

  bucket_name = "replace-with-your-unique-bucket-name"
```

```
}
```

Save the file.

Before running Terraform: Replace `replace-with-your-unique-bucket-name` with a lowercase, globally unique bucket name, such as `karthik-terraform-storage-2026-unique123` if available. If that name is already taken, choose another.

The AMI data source automatically looks up a matching Amazon Linux 2023 image in the configured AWS region.

10. Step 8: Add Code to Root `variables.tf`

For this version, the root configuration does not require additional input variables. You can leave the root `variables.tf` file empty.

The module-specific variables are already defined in their respective folders.

11. Step 9: Add Code to Root `outputs.tf`

Open the root `outputs.tf` file and paste:

```
output "vpc_id" {
  value = module.vpc.vpc_id
}

output "subnet_id" {
  value = module.vpc.subnet_id
}

output "ec2_instance_id" {
  value = module.ec2.instance_id
}

output "ec2_public_ip" {
  value = module.ec2.public_ip
}

output "s3_bucket_name" {
  value = module.s3.bucket_name
}
```

Save the file.

Purpose: Displays the resource IDs and S3 bucket name after Terraform creates the infrastructure.

12. Step 10: Run Terraform Commands in VS Code

In VS Code, click Terminal → New Terminal.

Make sure the terminal is in the project folder. If necessary, run:

cd C:\terraform-modules

Confirm Terraform is installed:

terraform -version

Check that the AWS CLI is configured:

aws sts get-caller-identity

If the AWS command fails, configure your AWS credentials before proceeding.

Run these Terraform commands in order:

A. Initialize Terraform

terraform init

Purpose: Downloads the AWS provider and initializes the modules.

B. Format the configuration

terraform fmt -recursive

Purpose: Formats the Terraform files consistently, including files inside module folders.

C. Validate the configuration

terraform validate

Expected output:

Success! The configuration is valid.

D. Preview the resources

terraform plan

Purpose: Shows the proposed infrastructure changes without creating resources.

E. Create the resources

terraform apply

Review the plan. If everything is correct, type:

yes

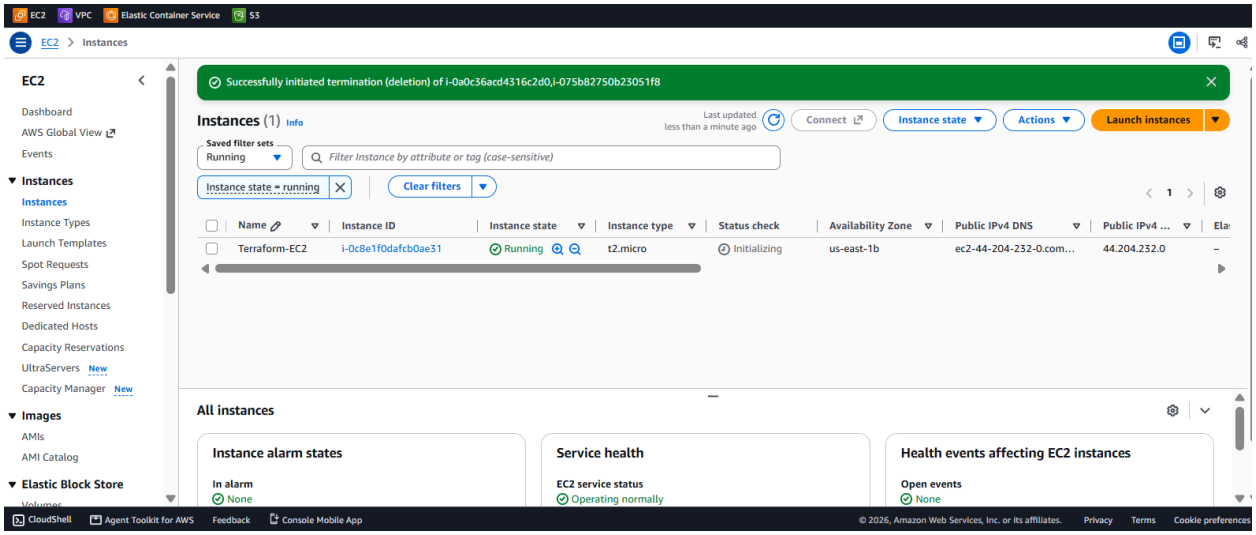
Terraform will create the resources if the AWS account has the necessary permissions and the configuration is valid.

F. Display the outputs

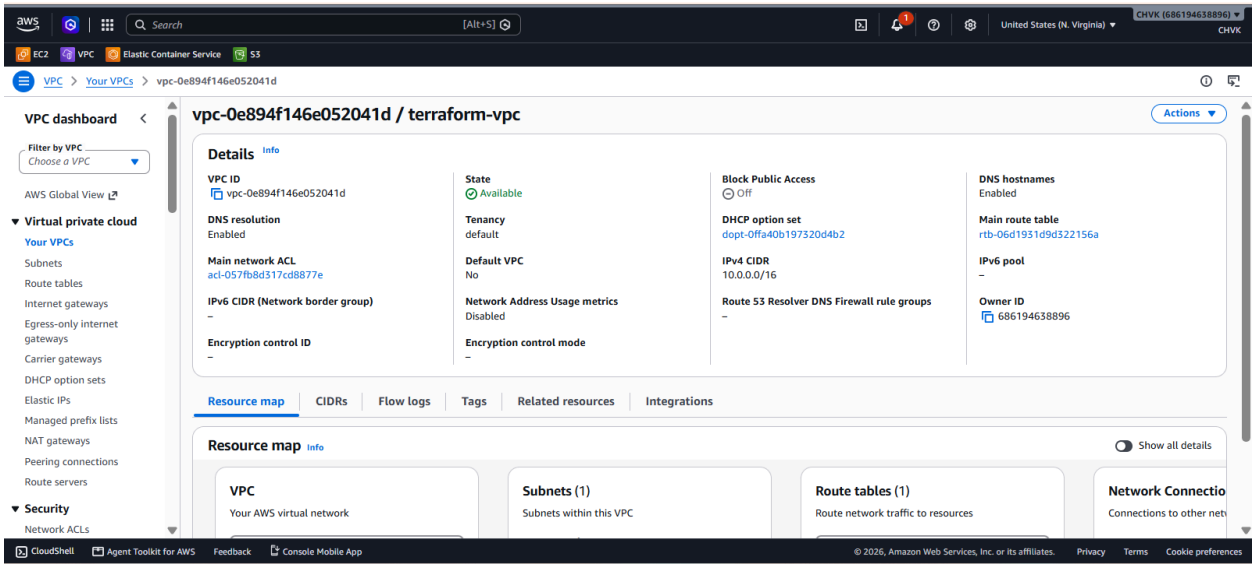
terraform output

```
PS C:\terraform-modules> terraform output
ec2_instance_id = "i-0c8e1f0dafcb0ae31"
ec2_public_ip = "44.204.232.0"
s3_bucket_name = "jagahesh-terraform-bucket"
subnet_id = "subnet-05869590181b061b3"
vpc_id = "vpc-0e894f146e052041d"
```

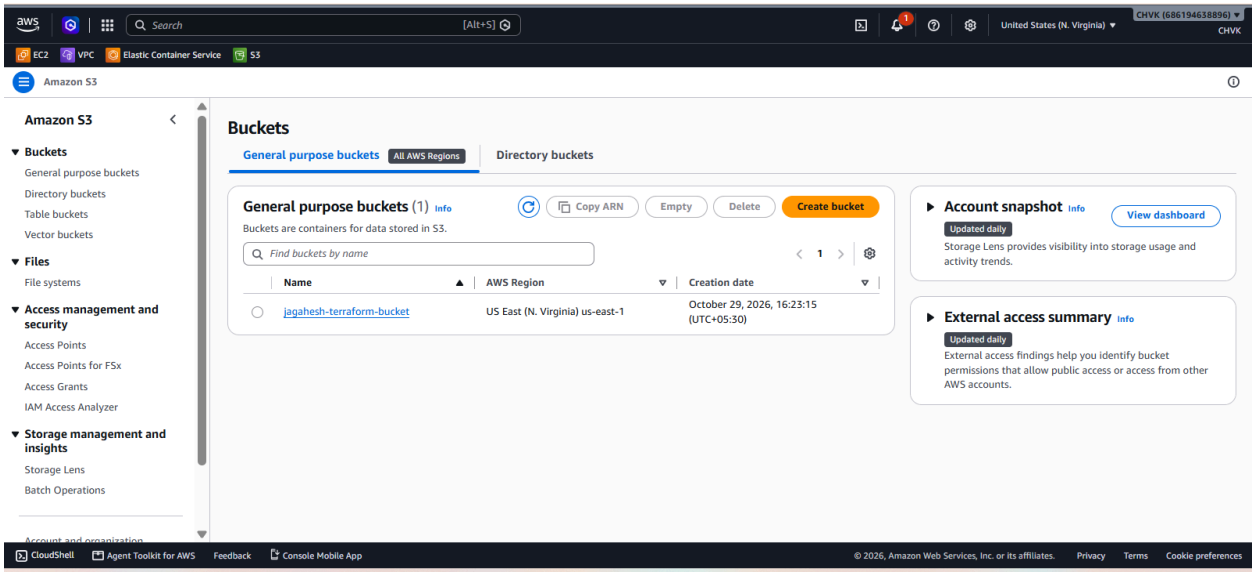
To check ec2 goto aws console search ec2 you see running instance



To check vpc goto aws console search vpc you see the vpc



To check s3 goto aws console search s3 you see the s3



You should see the VPC ID, subnet ID, EC2 instance ID, public IP address, and S3 bucket name.

Note that the subnet is not connected to an internet gateway in this example. A public IP alone does not make the EC2 instance internet-accessible. Add an internet gateway, public route table, and appropriate security rules if internet connectivity is required.

13. Result

Successfully created reusable Terraform modules for AWS VPC, EC2, and S3, and provisioned the resources through a single root configuration.

Task 3: Provision EC2 for Dev, Staging, and Prod Using Terraform Workspaces

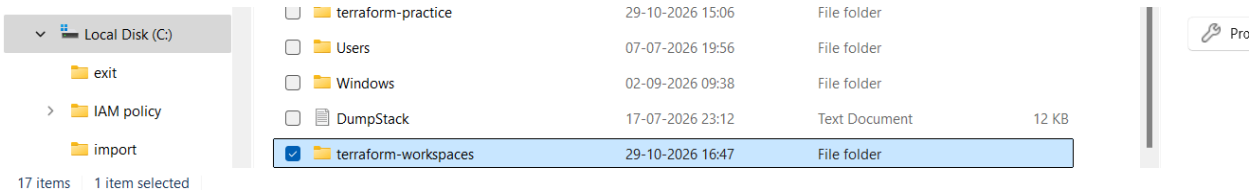
1. Objective

To create three separate EC2 instances for Dev, Staging, and Prod using Terraform workspaces and a single reusable Terraform configuration.

2. Step 1: Create a New Project Folder

We will create a separate project for this task.

1. Open File Explorer.
2. Open the `C:` drive.
3. Create a new folder named `terraform-workspaces`.
4. Open VS Code.
5. Click File → Open Folder.
6. Select `C:\terraform-workspaces`.
7. Click Select Folder.



Your project location is:

`C:\terraform-workspaces`

3. Step 2: Create the Required Files

In the VS Code Explorer panel, right-click the `terraform-workspaces` folder and select New File.

Create these three files:

- `providers.tf`
- `main.tf`
- `outputs.tf`

You do not need to create separate Dev, Staging, and Prod directories. Terraform workspaces maintain separate states for the same configuration.

Your folder structure should look like this:

```
C:\terraform-workspaces
│
├── providers.tf
├── main.tf
└── outputs.tf
```

4. Step 3: Configure `providers.tf`

Open `providers.tf` and paste:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Press Ctrl + S to save.

Purpose: Configures Terraform to use AWS in the `us-east-1` region.

5. Step 4: Configure `main.tf`

Open the root `main.tf` file and paste the following code:

```
# Find the latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
```

```
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Create EC2 for the selected workspace
resource "aws_instance" "server" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"

  tags = {
    Name        = "${terraform.workspace}-server"
    Environment = terraform.workspace
  }
}
```

Save the file.

Explanation

- `data "aws_ami"` finds a suitable Amazon Linux 2023 AMI.
- `aws_instance` defines the EC2 instance.
- `instance_type = "t2.micro"` specifies the EC2 instance size.
- `terraform.workspace` identifies the currently selected workspace.
- `Name` gives the instance a different name for each environment.
- `Environment` adds a tag identifying Dev, Staging, or Prod.

For example, the same code creates `Dev-server` when the active workspace is `Dev`, and `Staging-server` when the active workspace is `Staging`.

6. Step 5: Configure `outputs.tf`

Open `outputs.tf` and paste:

```
output "instance_id" {
  value = aws_instance.server.id
}

output "public_ip" {
  value = aws_instance.server.public_ip
}

output "environment" {
  value = terraform.workspace
}
```

Save the file.

Purpose: Displays the EC2 instance ID, public IP address, and environment name for the active workspace.

7. Step 6: Open the VS Code Terminal

Click Terminal → New Terminal.

Run:

cd C:\terraform-workspaces

Initialize Terraform:

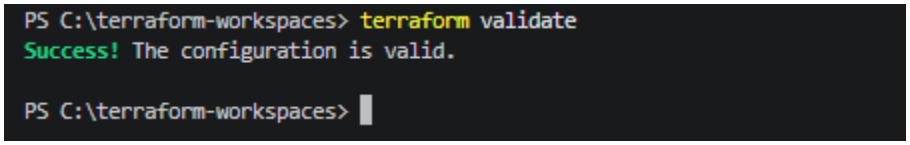
terraform init

Validate the configuration:

terraform validate

Expected validation output:

Success! The configuration is valid.



You can now create the workspaces.

8. Step 7: Create the Dev Workspace

Run:

terraform workspace new Dev

This command creates the `Dev` workspace and selects it.

Check the active workspace:

terraform workspace show

Expected output:

Dev

Preview the changes:

terraform plan

Create the Dev EC2 instance:

terraform apply

Review the plan and type `yes` when prompted.

Terraform creates the instance with the name `Dev-server` and the tag `Environment = Dev`.

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

environment = "Dev"
instance_id = "i-021e9d4451b4ffb2e"
public_ip = "3.95.14.22"
```

Check the output:

terraform output

```
PS C:\terraform-workspaces> terraform output
environment = "Dev"
instance_id = "i-021e9d4451b4ffb2e"
public_ip = "3.95.14.22"
```

9. Step 8: Create the Staging Workspace

Create the workspace:

terraform workspace new Staging

Check the workspace:

terraform workspace show

Expected output:

Staging

Preview the changes:

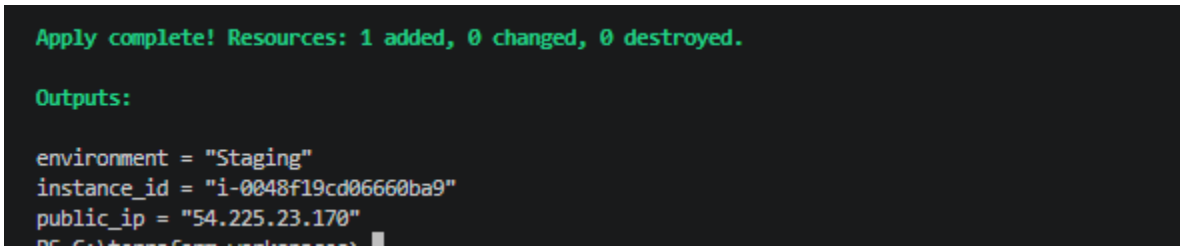
terraform plan

Create the Staging EC2 instance:

terraform apply

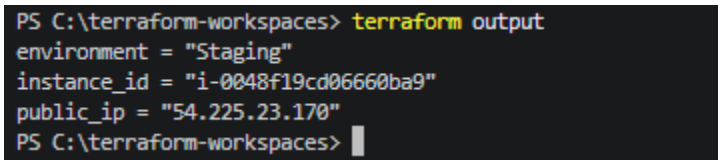
Review the plan and type `yes`.

Terraform creates a separate EC2 instance named `Staging-server`.



Check the output:

terraform output



10. Step 9: Create the Prod Workspace

Create the workspace:

terraform workspace new Prod

Check the workspace:

terraform workspace show

Expected output:

Prod

Preview the changes:

terraform plan

Create the Prod EC2 instance:

terraform apply

Review the plan carefully and type `yes` only if it is correct.

Terraform creates a separate EC2 instance named `Prod-server`.

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

environment = "Prod"
instance_id = "i-0729ce58555ff61cf"
public_ip = "13.217.224.70"
PS C:\terraform-workspaces>
```

Check the output:

terraform output

```
PS C:\terraform-workspaces> terraform output
environment = "Prod"
instance_id = "i-0729ce58555ff61cf"
public_ip = "13.217.224.70"
PS C:\terraform-workspaces>
```

Important: This is a basic learning example. For a real production environment, use suitable instance sizing, network security, monitoring, backups, and access controls. Terraform workspaces separate state, but they do not independently secure or isolate the AWS resources.

11. Step 10: List All Workspaces

Run:

terraform workspace list

Expected output:

default
 Dev
* Prod
Staging

```
PS C:\terraform-workspaces> terraform workspace list
  default
  Dev
* Prod
  Staging

PS C:\terraform-workspaces>
```

The order can differ. The * indicates the currently selected workspace.

12. Step 11: Switch Between Environments

To switch to Dev:

terraform workspace select Dev

To switch to Staging:

terraform workspace select Staging

To switch to Prod:

terraform workspace select Prod

After selecting a workspace, run:

terraform workspace show
terraform output

```
PS C:\terraform-workspaces> terraform workspace select Dev
Switched to workspace "Dev".
PS C:\terraform-workspaces> terraform workspace select Staging
Switched to workspace "Staging".
PS C:\terraform-workspaces> terraform workspace select Prod
Switched to workspace "Prod".
PS C:\terraform-workspaces> terraform workspace show
Prod
PS C:\terraform-workspaces> terraform output
environment = "Prod"
instance_id = "i-0729ce58555ff61cf"
public_ip = "13.217.224.70"
PS C:\terraform-workspaces>
```

Terraform uses the state associated with the selected workspace. Therefore, `terraform output` displays information only for that environment.

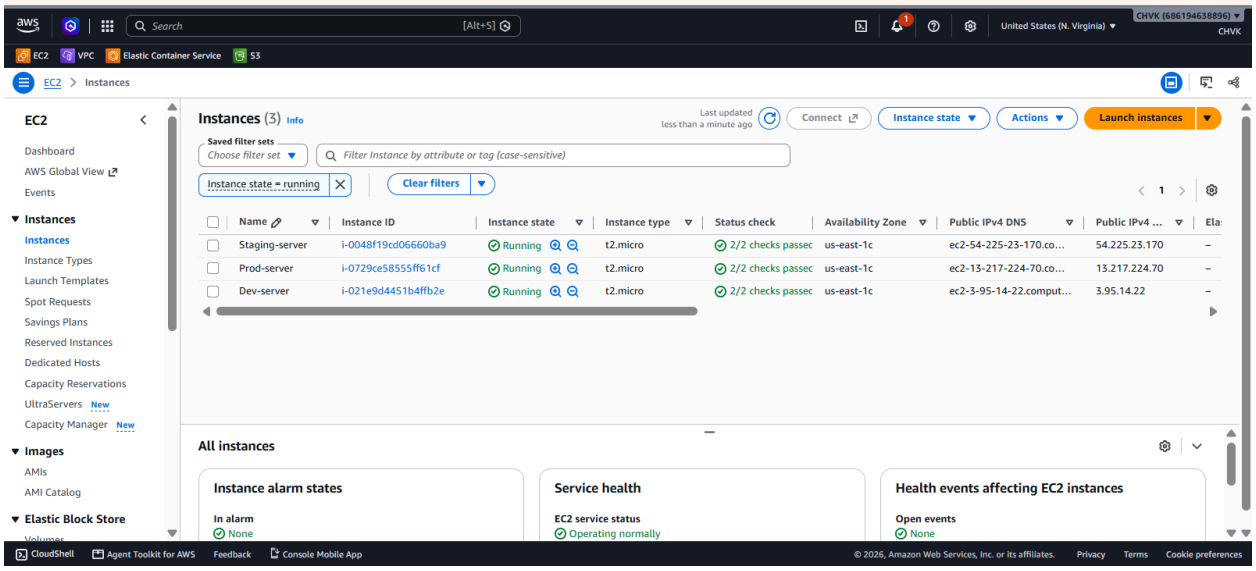
13. Step 12: Verify in AWS Console

1. Sign in to the AWS Management Console.
2. Open EC2.
3. Select Instances from the navigation panel.
4. Check that the three instances exist.
5. Verify their names and `Environment` tags.

You should see instances corresponding to:

Environment	EC2 instance name	Environment tag
Dev	`Dev-server`	`Dev`
Staging	`Staging-server`	`Staging`
Prod	`Prod-server`	`Prod`

These instances are created independently, but all use the same Terraform configuration.



14. Step 13: Destroy the Resources After Practice

Only perform this step if you no longer need the instances.

First, select Dev:

terraform workspace select Dev
terraform destroy

Select Staging:

terraform workspace select Staging
terraform destroy

Select Prod:

terraform workspace select Prod
terraform destroy

Review each proposed deletion and type `yes` only when you intend to delete the selected environment's resources.

Warning: Destroying Prod deletes the managed production instance. Do not run that command if you need to keep it running.

15. Quick Identification Table

Topic	Main command or symbol	Purpose
Terraform module	`module`	Reuses infrastructure code
Terraform workspace	`terraform.workspace`	Identifies the active workspace
Create workspace	`terraform workspace new`	Creates and selects a workspace
Switch workspace	`terraform workspace select`	Selects an existing workspace
List workspaces	`terraform workspace list`	Displays all workspaces
Preview changes	`terraform plan`	Shows proposed changes
Create resources	`terraform apply`	Creates or updates infrastructure
Display outputs	`terraform output`	Shows outputs for the active workspace
Delete resources	`terraform destroy`	Deletes resources managed by the active workspace

16. Final Result

Task 2: Created reusable Terraform modules for provisioning AWS VPC, EC2, and S3 resources.

Task 3: Used Terraform workspaces to create and manage separate EC2 instances for Dev, Staging, and Prod environments with one Terraform configuration.

Difference: Modules organize and reuse infrastructure code, while workspaces maintain separate Terraform states for different environments.